CTF Agent Benchmarking: Final Project Part 1

Team: Paul Prommark, Supawich Puengdang. **Repo:** `Puminunp/ctf-agent`. **Model:** `claude-haiku-5-5` via the Duke AI Gateway. **Benchmark:** 10 CTFTiny challenges (commit `f1c9531`). **Configs:** baseline `agent` (tag `frozen-eval-v1`, commit `7082312`); selected `agent-fix` (commit `2619f77`).

1. Summary

The frozen baseline solved 24 of 30 scored attempts. All 6 failures were HTTP 429 rate limits from the gateway, not agent mistakes. We changed only the retry policy and re-ran all 30 attempts: 30 of 30 solved (+20.0 pp), at +44% wall-clock time and +16% tokens. Neither run submitted a wrong flag.

2. Agent and controls

A trusted controller runs the loop: the model sees the challenge description, the contestant file list and the target name, calls a tool, sees the result, and decides the next step. The controller enforces the limits, which the model cannot change: 900 s, 60 steps and 400,000 tokens per attempt, 5 wrong flags, 3 turns without a tool call, 60 s per tool call.

Tool	Input	Output	Validation, timeout, errors
`run_shell`	bash command	exit code + output, cut at 8,000 chars	non-empty, ≤ 4,000 chars; killed in the container at the timeout; errors return as text
`read_file`	path	file text, capped	path must stay inside `/workspace`
`submit_flag`	candidate flag	"correct" or "incorrect" only	one line, ≤ 200 chars; checked in the controller by hash comparison

Each attempt gets a fresh container: unprivileged user, no sudo, all capabilities dropped, no Docker socket, no published ports, an internal network shared only with the target (no network for file-only challenges). Only the files listed in `challenge.json` are copied in; flags, solvers and write-ups stay in the controller. `scripts/check_isolation.py` confirmed from inside the sandbox, on 4 challenges, that the internet and host were unreachable and that no answer files were present (`runs/controls/`). A command tripwire for foreign hosts fired 0 times. The API key stays in the controller and is redacted from traces.

3. Evaluation protocol

Challenges. Very Easy: `cry-hybrid2`, `rev-whataxor`, `rev-unvirtualization`. Easy: `cry-babycrypto`, `rev-rap`, `for-1black0white`, `web-smug-dino`. Moderate: `cry-collision-course`, `pwn-bigboy`. Hard: `rev-maze`. That is five categories and four levels. `msc-showdown` was skipped because it needs a privileged container. Development used four other challenges (`rev-checker`, `rev-beleaf`, `rev-tablez`, `pwn-puffin`), all solved.

Runs. Trial 1 of all 10, then trial 2, then trial 3, each in a fresh container with a fresh conversation. Every attempt, including failures, is in `runs/<config>/results.jsonl` with a full trace. Each run used one clean commit. The config requests temperature 0.0, but the Duke gateway appears to drop that setting before calling Claude Haiku 5.5: in a direct test on November 11, Supawich found it accepts even an impossible temperature of 3.5. So the model probably ran at its default sampling, and runs were not repeatable step by step.

Metrics. Per challenge: solves out of 3, success rate, sample SD of the three 0/1 outcomes, and mean tokens and seconds. Standard pass@k averages $1 - C(3-c, k)/C(3, k)$ over challenges, where c = solves; its pass@1 equals the success rate. We also show "within the first k trials", which counts a challenge if any of its first k trials, in run order, solved it. Cost uses $0.05 in and $0.25 out per million tokens (Duke's price for Haiku 5.5); at Anthropic's list price ($0.10 in, $0.50 out) the same runs would cost $0.197 and $0.235.

4. Results

Challenge	Level	`agent` trials	Solved	SD (pp)	Tokens	Sec	`agent-fix` solved	Tokens	Sec
cry-hybrid2	VE	✓ ✓ 429	2/3	57.7	16,891*	9.6	3/3	16,006	11.7
rev-whataxor	VE	✓ ✓ ✓	3/3	0	42,048	18.9	3/3	25,373	10.4
rev-unvirtualization	VE	✓ ✓ ✓	3/3	0	36,570	14.5	3/3	31,809	15.0
cry-babycrypto	E	✓ ✓ ✓	3/3	0	16,613	10.9	3/3	12,799	9.3
rev-rap	E	429 429 ✓	1/3	57.7	79,763	51.0	3/3	116,471	97.8
for-1black0white	E	429 ✓ ✓	2/3	57.7	44,317	53.0	3/3	46,930	65.5
web-smug-dino	E	429 ✓ ✓	2/3	57.7	25,266*	19.9	3/3	21,789	16.7
cry-collision-course	M	✓ ✓ ✓	3/3	0	13,591	18.3	3/3	15,292	21.0
pwn-bigboy	M	✓ ✓ ✓	3/3	0	20,293	20.4	3/3	18,085	19.1

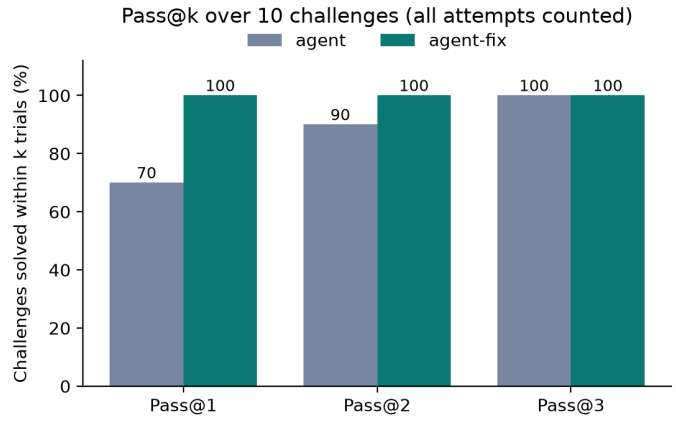
Challenge	Level	`agent` trials	Solved	SD (pp)	Tokens	Sec	`agent-fix` solved	Tokens	Sec
rev-maze	H	✓ 429 ✓	2/3	57.7	164,215	203.9	3/3	212,221	340.0

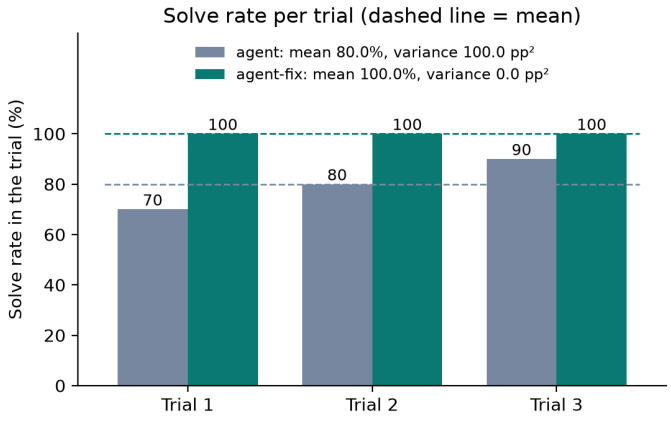
✓ = verified solve; 429 = ended on a rate limit. Tokens = mean input + output per attempt. `agent-fix` SD is 0 everywhere. *Average of 2 attempts; the third failed before any model reply, so it has no usage.

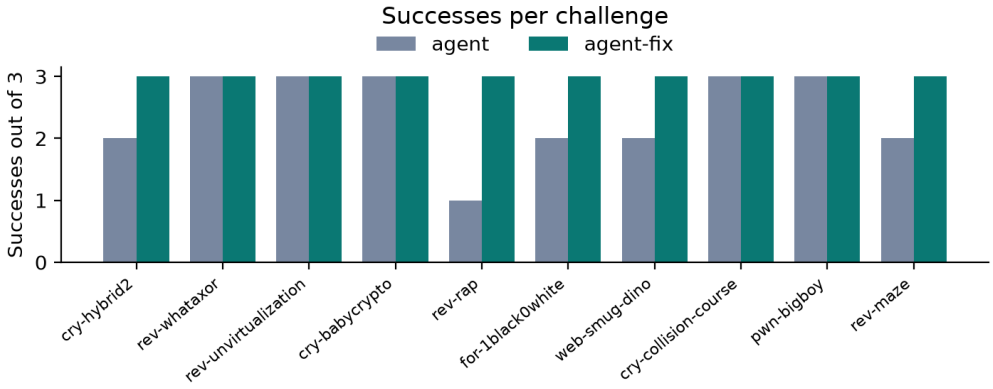
Metric (all attempts)	`agent`	`agent-fix`	Change
Solved / rate-limited / wrong flags	24 / 6 / 0	30 / 0 / 0	
Success rate	80.0%	100.0%	+20.0 pp
Pass@1 / 2 / 3 (standard)	80.0 / 96.7 / 100%	100 / 100 / 100%	+20.0 / +3.3 / 0 pp
Pass@1 / 2 / 3 (within first k trials)	70 / 90 / 100%	100 / 100 / 100%	+30 / +10 / 0 pp
Solve rate per run, mean ± SD	70 / 80 / 90%, 80.0 ± 10.0 pp	100%, ± 0	
Wall-clock total (per attempt)	1,261.5 s (42.1 s)	1,819.6 s (60.7 s)	+44%
Model / tool calls	201 / 202	225 / 225	
Tokens in / out	1,177,441 / 159,101	1,350,462 / 199,866	+16%
Cost (Duke price)	$0.099	$0.117	+$0.019

`rev-maze`, the only Hard challenge, took the most time and tokens: 11 to 22 steps per attempt and all 4 tool timeouts. Most other solves took 4 to 8 steps.

Figures (`runs/plots/`, made by `scripts/make_plots.py`): pass@k, solve rate per trial with mean and variance, and successes per challenge, for `agent` and `agent-fix`. The tokens and seconds per challenge are in the table above (the chart is `runs/plots/tokens_and_seconds_per_challenge.png`).







5. Problem, change and comparison

Problem. Six baseline attempts ended on HTTP 429: `rev-rap` t1 and t2, `for-1black0white` t1, `web-smug-dino` t1, `rev-maze` t2, `cry-hybrid2` t3. The client retried only twice (waiting 1 s, then 2 s) and ignored the gateway's `Retry-After` header.

Hypothesis. A 429 is temporary. Waiting longer should let those calls succeed with no other change.

Change. `agent-fix` changes only the retry settings: up to 6 retries, waits up to 60 s, honors `Retry-After`, at most 150 s of waiting per call. Model, prompt, tools and limits are identical.

Result. On the same 10 challenges × 3 fresh trials: 30 of 30 solved, +20.0 pp. The six lost slots all finished: `rev-rap` t1 and t2 in 10 and 12 steps, `for-1black0white` t1 in 8, `web-smug-dino` t1 in 5, `rev-maze` t2 in 19, `cry-hybrid2` t3 in 4. On the 24 slots solved in both runs, `agent-fix` used 5% fewer tokens but 34% more time (1,262 s vs 940 s), which fits waiting out rate limits.

Caveats. The gain is robustness, not solving skill: both configs hit 100% on attempts that ran to completion. The runs were on different days, so a quieter gateway may also have helped. Retries were not logged, so no trace step shows a 429 being absorbed.

6. Failure analysis and limitations

Annotated case: `agent-for-1black0white-t1`. Steps 1 to 3: the agent read `qr_code.txt` and recognized a QR code. Steps 4 to 7: with no QR library installed, it chose not to fetch one from the internet and wrote its own decoder. Steps 8 to 12: one character came out wrong, so it tried Reed-Solomon error correction, then single bit flips. On the 14th model call, about 90 s and 69,578 input tokens in, the gateway returned 429 on all three tries, and the attempt ended. The approach was sound; the loss was infrastructure. The same slot solved in 8 steps under `agent-fix`. **Testable improvements:** (1) log every retry as a trace event and count attempts lost to rate limits; (2) trim old tool output, since input per call grew from 1,398 to 10,512 tokens over these 13 steps.

Limitations.

- All 54 completed attempts were solved, so this set cannot show where solving fails.
- Only one small model was tested.
- Two earlier launches of the full `agent-fix` run were harness failures, not results, and are kept as incident records: Docker Desktop had stopped (30 records of 0.3 s, no tokens, `runs/fix_aborted_docker_down/`), and then the gateway was unreachable without the Duke VPN (7 records of about 210 s, no tokens, `runs/fix_aborted_network_down/`). The third launch, over the VPN, is the reported run. An earlier re-run of only the 6 rate-limited slots (`runs/rerun/`, 6 of 6 solved) is superseded by the full run.
- The token cap is checked before each call, so one attempt can overshoot it by one call.
- Five scored challenges ship x86-64 Linux binaries. The scored runs used Paul's Windows x86-64 laptop; on Apple Silicon the sandbox runs under x86 emulation.

7. Configuration for the competition

We keep `agent-fix` for November 18, launched with `scripts/run_attempts.py --config configs/agent-fix.json`. The trade-off is time: one call may now wait up to 150 s. Before the competition, not yet measured, we plan to:

- log retries and trim old tool output;
- try `claude-sonnet-5-5`, which is on the Duke gateway, on harder practice challenges, and count refusals.

8. Disclosures, contributions and AI use

Prior exposure. No scored challenge was run or read before the scored run. Three challenges from our first scored list (`rev-checker`, `rev-beleaf`, `pwn-puffin`) were run as pilots and replaced before scoring, so they are development challenges. The full `rev-checker` flag and part of the `rev-beleaf` flag appeared in a working chat while we read pilot traces. After the baseline run, a partial decoded string of the `for-1black0white` flag (a scored challenge) appeared in the same chat while we analysed a failure; it came after scoring and changed no result. In nearly all solves (52 to 53 of 54, depending on how they are counted) the exact flag appeared in tool output before submission; the rest assembled it from earlier output. CSAW write-ups are public, so the model may have seen these challenges before.

Contributions. Paul Prommark built the agent, controller, sandbox, tests and analysis scripts, and ran the pilots and both scored runs. Supawich Puengdang researched the benchmark, models and controls, re-checked every metric from the raw records (correcting pass@k, token and cost reporting), verified the cited trace steps, and wrote the report.

AI use and reuse. Code, tests and drafts were written with Claude Code (Anthropic), directed and reviewed by the team. We reused NYU CTF Bench and CTFTiny data, the `nyuctf` 1.1.1 loader, Duke's gateway guide, and the course Implementation Guide's control flow. Setup and launch commands are in `README.md`. The demo video shows one fresh, uncut `agent-fix` run on the practice challenge `rev-tablez` (`runs/demo_video/`, run ID `agent-fix-rev-tablez-t1`, commit `b7be43c`): solved in 11 steps and 48.9 s, with one wrong flag before the correct one. Paul recorded and narrated it on his Windows x86-64 laptop.